

Ledger Architecture Review — Technical ADR

Status: Proposed

Scope: Financial ledger implementation

Primary concerns: Correctness, scalability, lifecycle management, operational safety

1. Context

The implementation uses an immutable, append-only ledger as the system of record. This provides deterministic history, auditability and the ability to reconstruct financial state.

The design intentionally separates **financial history** from **derived operational state**. This is important because scaling the read path should not require compromising the immutability of the underlying ledger.

2. Decision: Append-Only Ledger at Scale

Decision

Retain the append-only ledger as the authoritative source of truth.

At higher volume, introduce:

- time/account-based partitioning;
- indexed access paths for active accounts;
- periodic balance snapshots/materialized state;
- archival of immutable historical partitions.

What breaks first at 100×?

The first thing that breaks at **100× volume is read performance, not necessarily writes**.

An append-only ledger can continue accepting entries efficiently, but operations that reconstruct a balance or authorization state become increasingly expensive if they have to scan or replay a growing transaction history.

The design accumulates unbounded state in the **ledger history itself**, along with its indexes and any derived state maintained from it. Financial history cannot simply be deleted, so this growth is inherent to the model.

The **cheapest structural change** is to introduce **periodic snapshots/materialized balances**:

Current state = latest snapshot + ledger entries since that snapshot

If a balance or authorization query requires scanning/replaying an increasingly large transaction history, latency and I/O grow with historical volume. Secondary indexes also become increasingly expensive to maintain and the list and used map will be memory consuming.

The unbounded state is:

Ledger history → indexes → derived/materialized state

The ledger itself must remain unbounded from a business perspective because financial history cannot simply be deleted when it becomes inconvenient.

Cheapest structural change

Introduce **periodic snapshots**.

Instead of:

Replay transaction 1 ... transaction N

a balance query becomes:

Latest snapshot + transactions after snapshot

This reduces the amount of historical data required for normal reads without changing the source-of-truth model.

Alternative considered

Full distributed/sharded ledger: rejected at this stage because it introduces substantially more complexity around ordering, consistency, cross-shard operations and recovery before the current volume requires it.

3. Decision: Value-Dated Entries

Technical distinction

The implementation must distinguish:

- **event/booking timestamp** — when the system records the operation;
- **value date** — when the financial effect becomes effective;
- **settlement timestamp** — when the financial obligation is actually settled.

Treating these as the same field would make future-dated and backdated operations difficult to control.

Production impact

Value-dating affects:

- available and accounting balances;
- reconciliation;
- accrual/interest calculations;
- reporting;
- end-of-day processing;
- cut-off handling;
- audit and investigation;
- correction and backdating workflows.

It also introduces a particularly important failure mode: **a technically valid write can produce an economically incorrect balance if the effective date is wrong** And also it s very confusing how the change when value_date and book date are different it will affect the balance of the the value_date and we need to update the balance off all days till the booked date specially if value_date< booked date .

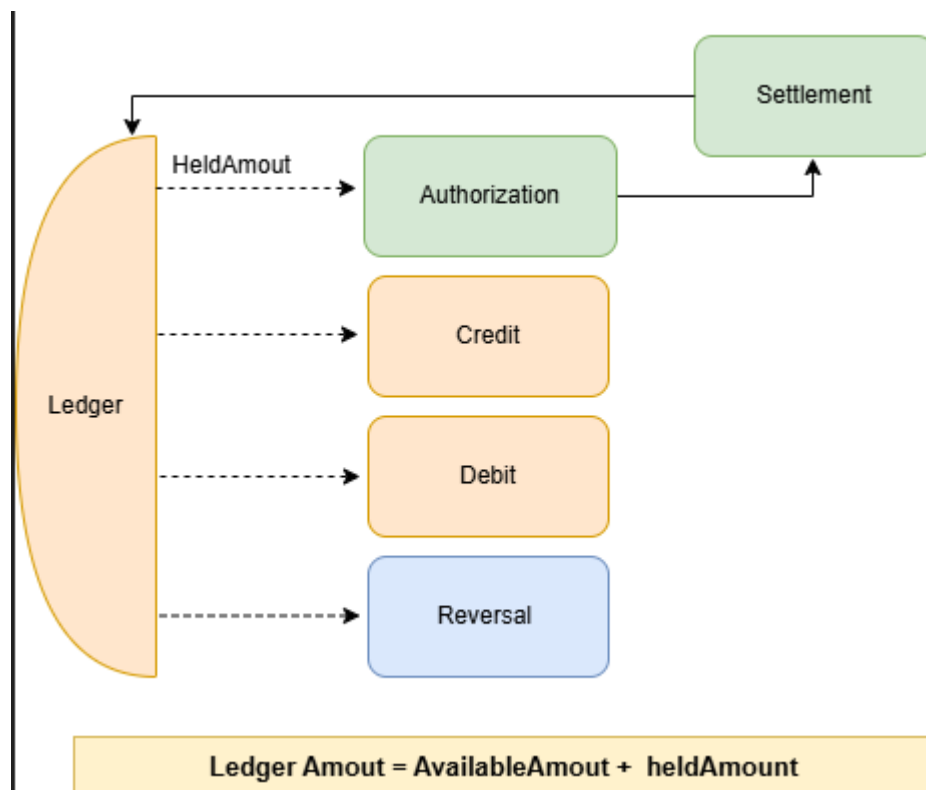
Case : E7 has:

booking day = Day 5

value date = Day 2

E7 changes the Day 2 ledger position to: -370 and the fee itself has.

and also for authorization :E5 → settlement Auth-A = 185



Go-live control

dual authorization for manual value-date changes.

The audit record should contain:

original value date → requested value date → requester → approver → reason → timestamps

need to add extra information related to the processing of the authorization.

The original ledger record must remain immutable.

Value-dating creates a separation between **when a transaction is recorded** and **when its financial effect takes place**. In a UAE-licensed bank, that affects more than the ledger: it impacts balance calculation, reconciliation, interest/accrual processing, end-of-day processing, reporting, audit and investigation.

From a regulatory perspective, the bank must maintain accurate accounting and transaction records with an audit trail and be able to reconstruct transactions. CBUAE requirements also require transaction records to be retained for at least five years in the relevant circumstances.

The main production risk is **unauthorized or incorrect backdating**: a technically valid entry could change the financial position for a prior period and potentially affect reporting, reconciliation or customer balances.

Control I would add before go-live:

Require **maker-checker approval for any manual value-date change**. The system would immutably record the original date, requested date, requester, approver, reason and timestamps, and prevent the original ledger entry from being modified.

This gives us both **technical traceability and segregation of duties**, which is consistent with CBUAE internal-control expectations.

4. Decision: Authorization State Machine

An authorization is not simply:

authorized → settled

The production lifecycle must account for terminal outcomes that do not produce a settlement.

Outcome	Example	Required behavior
Expiry	Authorization window expires	Automatically close and reject later settlement
Cancellation	Customer/merchant cancels	Mark terminal; retain reason and actor
Reversal	Previously accepted operation is undone	Append compensating transition
Decline	Risk/limit/control rejects transaction	Make non-settleable and retain decision evidence

Timeout	Downstream processor never completes	Move to explicit timeout/exception state
Account invalidation	Account becomes blocked/closed	Prevent settlement and route to exception handling
Supersession	Corrected authorization replaces original	Link records and make original non-settleable

Key invariant

No terminal transition deletes the original authorization.

This prevents a particularly dangerous production failure: an old authorization becoming accidentally settleable after its business validity has ended.

5. Concurrency and Idempotency

The simplified implementation does not attempt to solve every production concurrency scenario. neither the scaling or case of hundred of Event and hundred accounts.

This will result in serious issue on the memory and management of the account and the ledger balance and fee and interest .

At production scale, the critical requirement is that retries cannot create duplicate financial effects.

For example:

`request → authorization → network timeout → client retry`

must not produce:

`two authorizations / two settlements`

The production design therefore requires:

- idempotency keys;
- atomic state transitions;
- optimistic/pessimistic concurrency control where appropriate;

- unique constraints for externally supplied transaction identifiers;
- deterministic handling of duplicate requests.

The ledger remains append-only, but **state-transition validation must be atomic**.

6. What Was Deliberately Cut

Simplification	Reason	Deferred production risk
Single logical storage model	Keep accounting semantics clear	Sharding and horizontal scaling
Basic lifecycle	Keep core domain manageable	Timeouts, reversals and operational exceptions
Simplified value dating	Avoid banking-calendar complexity	Cut-offs, holidays and controlled backdating
No reconciliation engine	Outside core ledger responsibility	Cross-system financial discrepancies
No multi-region active/active	Avoid premature distributed complexity	Regional failure and availability
Basic concurrency model	Focus on ledger semantics	High-contention accounts

These are **scope decisions, not assumptions that the problems do not exist**.

7. Production Architecture Evolution

The intended evolution is:

Phase 1 — Correctness

Immutable ledger + deterministic transitions + audit trail.

↓

Phase 2 — Operational scale

Partitioning + indexes + snapshots + idempotency + reconciliation.

↓

Phase 3 — Enterprise resilience

Archival/retention + multi-region recovery + operational controls + regulatory integrations.

This sequencing avoids introducing distributed-system complexity before it solves an actual bottleneck.

8. Key Engineering Trade-Off

The central architectural trade-off is:

Preserve the ledger's immutable source of truth, while allowing derived state to be optimized aggressively for read performance.

Snapshots, indexes and materialized views can be rebuilt from the ledger. The ledger itself therefore remains the authoritative recovery mechanism.

That gives the system a useful property for a financial platform:

performance optimizations can fail or be discarded without destroying the underlying financial history. for that we need to think about publishing those event to kafka and decide after if we want to store those event on db or just do kind of caching to redis.

9. Production Gate

I would not consider the implementation production-ready until the following are demonstrated:

- atomic/idempotent financial operations;
- complete authorization terminal-state handling;
- controlled value-date modification;
- reconciliation with external systems;
- partitioning/snapshot strategy;
- recovery from partial failures and retries;
- access control and segregation of duties;

- monitoring for invariant violations and reconciliation breaks.

Architecture conclusion: The current implementation is a suitable correctness-focused ledger core. Production hardening should extend around it rather than weaken the append-only source-of-truth model.

The current solution i implement does not handle the interest neither the fee because of the complexity of the algorithm